# Relay — Deployment Runbook

A single-workspace Slack bot that routes messages to multiple Claude Managed Agents (CMAs) by slug. This runbook covers everything from provisioning a fresh AWS Lightsail instance to managing agents day-to-day. Pairs with the repo at https://github.com/shanerlevy-debug/Relay (the `README.md` there is the developer-side reference).

**Version:** 1.0 — May 2026 **Audience:** Mixed. The deploy sections are for whoever stands up the box. The agent-editing and operations sections are for whoever runs the bot day-to-day.

---

## 1. What Relay is, in one paragraph

Slack and Anthropic's Claude Managed Agents API don't speak the same wire protocol — Slack pushes events to an HTTPS endpoint or holds a WebSocket open, while a CMA is an invokable session that you POST to. Relay is the smallest reasonable process that translates between the two. It runs a Slack Bolt client in Socket Mode (outbound WebSocket, no public URL), accepts `@mentions` and a `/ask` slash command, parses an *agent slug* out of each message, spins up a CMA session against the corresponding agent, streams the reply back into Slack, and persists thread state in SQLite so follow-up messages keep talking to the same agent. About 280 lines of Python, $3.50/mo of compute.

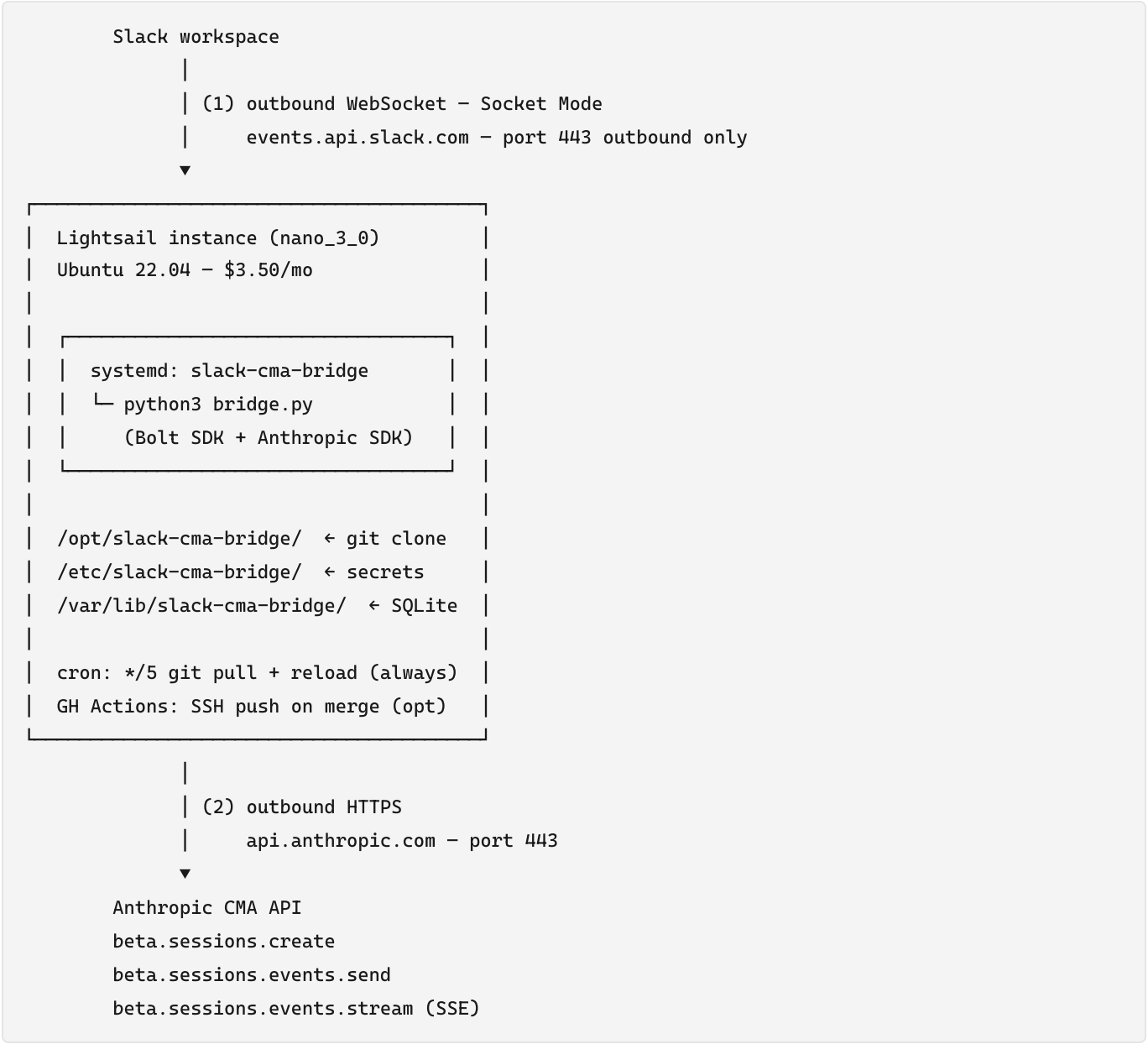
## 2. What Relay is **not**

- **Not multi-tenant.** One Slack workspace, one process. To support N workspaces you would need to multiply by N, plus build an OAuth install flow.
- **No identity.** Every Slack user looks the same to the bridge — no RBAC, no per-user permissions, no audit log of who said what.
- **No web UI.** Agent registry is YAML in this repo. Changes happen via `git push`, not a settings page.
- **No production hardening beyond the basics.** systemd `Restart=always`, hardened service unit, SQLite WAL — that's it. No monitoring, no metrics, no alerting, no backups.
- **No SLA.** If the bot stops responding, you find out when someone Slacks the test channel.

If you need any of the above, see §11.

---

## 3. Architecture




**Inbound to the box:** SSH (port 22) only. The bridge needs no public ports.

**Outbound from the box:** Slack WebSocket (events.api.slack.com:443), Anthropic API (api.anthropic.com:443), GitHub for repo pulls (github.com:443).

### Why Socket Mode

The alternative — Slack POSTing events to a public URL — would require running an HTTPS server on the box, terminating TLS, validating Slack's `X-Slack-Signature` HMAC, handling the 3-second response deadline, and managing the `url_verification` handshake. Socket Mode collapses all of that into one outbound WebSocket connection. Corporate firewalls usually allow this; if yours doesn't, see §11.

## 4. Prerequisites

Before you begin, gather:

| Item | Where to get it |
|---|---|
| AWS account with Lightsail enabled | console.aws.amazon.com |
| Slack workspace admin access | The workspace you're deploying to |
| Anthropic API key with CMA beta access | console.anthropic.com → API Keys |
| At least 2 CMAs created in Anthropic Console | console.anthropic.com → Managed Agents → New Agent. Note each one's `agent_…` ID. |
| A CMA environment ID | console.anthropic.com → Managed Agents → Environments. Note the `env_…` ID. |
| Git client and SSH client on your local machine | OpenSSH on Windows 10+/macOS, openssh-client on Linux |

You will need three secrets when configuring the box:

- `xoxb-…` — Slack **Bot User OAuth Token** (per workspace, after app install)
- `xapp-…` — Slack **App-Level Token** with `connections:write` scope
- `sk-ant-…` — Anthropic **API key**

Generate these in §6.

---

## 5. AWS Lightsail provisioning

This section produces a running Ubuntu 22.04 instance with SSH access. Total time: ~5 minutes.

### 5.1 Create the instance

**Option A — AWS console (clickable):** 1. Sign in to https://lightsail.aws.amazon.com. 2. Pick a region near you. `us-east-1` is fine. 3. **Create instance** → Linux/Unix → **Ubuntu 22.04 LTS**. 4. **Instance plan** → **$3.50 USD** (512 MB RAM, 2 vCPUs, 20 GB SSD). One vCPU is plenty; the bridge sits at ~50 MB RAM idle. 5. **SSH key pair** → either let Lightsail auto-generate (download the `.pem` immediately — Lightsail doesn't show it again) or upload your own public key. 6. **Identify your instance** → name it `relay-bridge`. 7. **Create instance** and wait ~30 seconds for state to reach `running`.

**Option B — AWS CLI** (if you have it installed and configured):

```
# Generate a keypair locally
ssh-keygen -t ed25519 -f .lightsail/relay -N '""' -C "relay-bridge"

# Import the public key to Lightsail
$pub = Get-Content .lightsail/relay.pub -Raw
aws lightsail import-key-pair --region us-east-1 `
    --key-pair-name relay-bridge-key --public-key-base64 "$pub"

# Create the instance
aws lightsail create-instances --region us-east-1 `
    --instance-names relay-bridge `
    --availability-zone us-east-1a `
    --blueprint-id ubuntu_22_04 `
    --bundle-id nano_3_0 `
    --key-pair-name relay-bridge-key

# Get the public IP
aws lightsail get-instance --region us-east-1 --instance-name relay-bridge `
    --query 'instance.publicIpAddress' --output text
```

### 5.2 Lock down SSH key permissions (Windows)

On Windows the SSH client refuses to use a key with loose permissions. Fix once:

```
icacls .lightsail\relay /inheritance:r /grant:r "${env:USERNAME}:F"
```

On Linux/macOS: `chmod 600 .lightsail/relay`.

### 5.3 Verify SSH

```
ssh -i .lightsail/relay ubuntu@<PUBLIC-IP>
```

You should land at `ubuntu@ip-…:~$`. Type `exit` to disconnect.

---

## 6. Slack app creation

This section produces the `xoxb-` and `xapp-` tokens.

### 6.1 Create the app from manifest

1. Open https://api.slack.com/apps in your demo workspace.

2. **Create New App** → **From a manifest** → choose your workspace.
3. Paste the contents of `slack-app-manifest.yaml` from this repo. Review and create.

### 6.2 Generate the App-Level Token

1. In the new app's settings: **Basic Information** → **App-Level Tokens** → **Generate Token and Scopes**.
2. Scope: `connections:write`. Name: anything (`relay-socket`).
3. Click **Generate**. Copy the `xapp-…` token — this is `SLACK_APP_TOKEN`.

### 6.3 Install to workspace

1. Left sidebar: **Install App** → **Install to Workspace** → **Allow**.
2. On the next screen, copy the **Bot User OAuth Token** (`xoxb-…`) — this is `SLACK_BOT_TOKEN`.

### 6.4 Upload an icon (optional)

**Basic Information** → **Display Information** → **App icon** → **upload** the `relay-logo-512.png` from this repo (or your own).

### 6.5 Invite the bot to a channel

In Slack, in any channel: `/invite @Relay`.

---

## 7. Bridge deployment

Now provision the box itself. Total time: ~3 minutes.

### 7.1 Run bootstrap

SSH into the Lightsail instance:

```
ssh -i .lightsail/relay ubuntu@<PUBLIC-IP>
```

On the box:

```
sudo git clone https://github.com/shanerlevy-debug/Relay.git /opt/slack-cma-bridge
cd /opt/slack-cma-bridge
sudo bash deploy/bootstrap.sh
```

`bootstrap.sh` is idempotent — safe to re-run. It:

1. Installs `python3-venv` and `git` via `apt`.

2. Creates the `slackbridge` system user.
3. Clones the repo to `/opt/slack-cma-bridge` (if not already present).
4. Builds a venv at `/opt/slack-cma-bridge/.venv` and installs `requirements.txt`.
5. Creates `/etc/slack-cma-bridge/` (chmod 750) and `/var/lib/slack-cma-bridge/`.
6. Writes a placeholder `/etc/slack-cma-bridge/bridge.env` (chmod 640, root:slackbridge) with all the env vars the bridge needs.
7. Installs the systemd unit at `/etc/systemd/system/slack-cma-bridge.service`.
8. Installs `/etc/cron.d/slack-cma-bridge-redeploy` which runs `redeploy.sh` every 5 minutes.

### 7.2 Install your secrets

Edit the placeholder env file with your three tokens:

```
sudo vi /etc/slack-cma-bridge/bridge.env
```

Replace the three `…replace-me` values:

```
SLACK_BOT_TOKEN=xoxb-real-token-here
SLACK_APP_TOKEN=xapp-real-token-here
ANTHROPIC_API_KEY=sk-ant-real-token-here

AGENTS_CONFIG=/opt/slack-cma-bridge/agents.yaml
DB_PATH=/var/lib/slack-cma-bridge/threads.db
SLASH_COMMAND=/ask
LOG_EVENT_TYPES=0
```

Save and exit. The file should remain mode 640, root:slackbridge — `bootstrap.sh` set that already.

### 7.3 Configure agents

```
cat /opt/slack-cma-bridge/agents.yaml
```

Verify it has your real `environment_id` and at least one agent with a real `anthropic_agent_id`. If the values shipped in the repo are not yours (likely), edit `agents.yaml` **locally** in your fork, commit, push — then on the box `sudo /opt/slack-cma-bridge/deploy/redeploy.sh` to pull immediately, or wait up to 5 min for cron.

### 7.4 Start the bridge

```
sudo systemctl enable --now slack-cma-bridge
sudo journalctl -u slack-cma-bridge -f
```

You should see, within a few seconds:

```
relay starting; 2 agents loaded; default=slack-test; slugs=['slack-test', 'vanguard']
slack_bolt.App ⚡ Bolt app is running!
```

In the Slack app's **Settings** → **Socket Mode** page, the connection indicator turns green.

### 7.5 Smoke test from Slack

In a channel where the bot is invited:

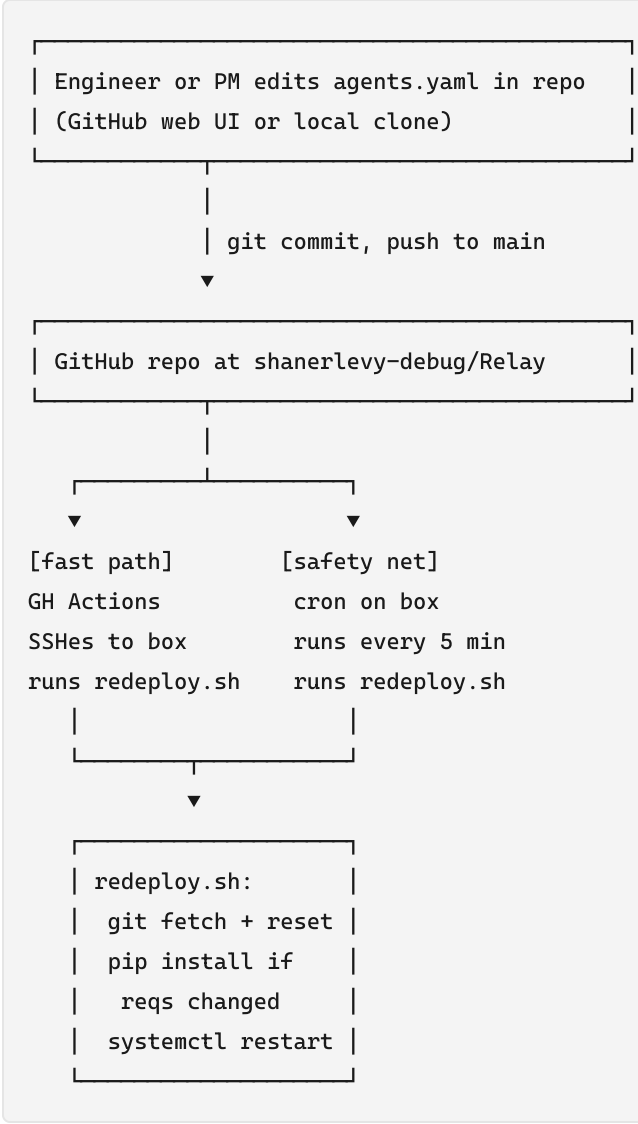
| What to type | Expected |
|---|---|
| `@Relay slack-test ping` | Within ~5 seconds, the bot replies in a thread |
| In that thread, reply *without* re-tagging: `tell me more` | Same agent answers, with memory of the prior turn |
| `@Relay vanguard <a research question>` | Routes to Vanguard CMA, replies in a new thread |
| `/ask slack-test who founded Anthropic?` | Slash command works; bot replies top-level (not threaded) |
| `@Relay hello there` (no slug) | Routes to the *default* agent |

Failure mode reference: see §10.

---

## 8. Agent management (git-ops)

Once deployed, **never** edit `agents.yaml` directly on the box. Local edits get wiped on the next `git pull`. The full workflow:




## 8.1 Add or remove an agent

1. Edit `agents.yaml` (locally or in GitHub's web UI):

```yaml environment_id: env_018YFpVVwTNWu8TRef2jfbvq default: slack-test

agents: slack-test: anthropic_agent_id: agent_0112NyBraSMpk7Tjub9mMAWh description: Test bot. Default for untagged messages.

```
vanguard:
  anthropic_agent_id: agent_011CaPqFYYtWssBNheHKPA5g
  description: Market research bot.

coder:                                  # ← new
  anthropic_agent_id: agent_replace_me
  description: Code review and implementation help.
```
```

```

2. Commit and push to `main`.
3. Within 5 minutes the cron job applies the change. To trigger immediately: - **Option A — wire the fast path** (do once): in repo Settings → Secrets and variables → Actions, add `LIGHTSAIL_HOST`, `LIGHTSAIL_USER` (= `ubuntu`), `LIGHTSAIL_SSH_KEY` (= contents of your private key file). On the next merge the GH Action SSHes in and reloads instantly. Without these secrets the workflow no-ops cleanly. - **Option B — manual trigger**: SSH in and run `sudo /opt/slack-cma-bridge/deploy/redeploy.sh`.

### 8.2 Rename a slug

Same as adding — edit and push. Active Slack threads pinned to the old slug name will break on the next reply (they'll route to the default agent because the old slug no longer matches). Telling people about renames before pushing avoids confusion.

### 8.3 Change the default agent

Change the `default:` line at the top of `agents.yaml`.

### 8.4 What does NOT require a redeploy

Changes inside a CMA's *own* configuration (system prompt, tools, skills) — those happen in the Anthropic Console and apply to the next session you create. You don't need to touch this repo.

---

## 9. Operations

### 9.1 Tailing logs

```
ssh -i .lightsail/relay ubuntu@<IP> "sudo journalctl -u slack-cma-bridge -f"
```

For the redeploy log:

```
ssh -i .lightsail/relay ubuntu@<IP> "sudo tail -f /var/log/slack-cma-bridge-redeploy.log"
```

### 9.2 Restarting

```
sudo systemctl restart slack-cma-bridge
```

systemd's `Restart=always` already covers the bridge crashing. You'd manually restart only after editing `bridge.env` (which doesn't auto-reload).
```

### 9.3 Stopping (temporarily)

```
sudo systemctl stop slack-cma-bridge
```

`disable` prevents auto-start on boot; usually not what you want.

### 9.4 Updating bridge code or dependencies

Same as agent changes — push to `main`. If `requirements.txt` changed, `redeploy.sh` reinstalls automatically.

### 9.5 SSH key rotation

1. Generate a new keypair locally.
2. Lightsail: **Networking** → **SSH keys** → **upload** the new public key.
3. Append the new public key to `~/.ssh/authorized_keys` on the box (over the existing SSH session before logging out).
4. Update GH Actions secret `LIGHTSAIL_SSH_KEY` if you've wired the fast path.
5. Verify SSH works with the new key, then remove the old one.

### 9.6 Backing up thread history

`/var/lib/slack-cma-bridge/threads.db` holds the slug-pin and session-id mapping per Slack thread. Losing it means active threads "forget" which agent they were talking to (next reply routes to the default). For a demo this is acceptable. To persist:

```
sudo sqlite3 /var/lib/slack-cma-bridge/threads.db ".backup '/tmp/threads-$(date +%Y%m%d).db'"
```

Schedule via cron and upload to S3 if you care.

### 9.7 Snapshot the instance

Lightsail console → **Snapshots** → **Create instance snapshot**. Each costs $0.05/GB-month while retained. Useful before significant changes.

---

## 10. Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| Service doesn't start, `journalctl` shows `agents config not found` | `AGENTS_CONFIG` points at a missing path | `cat /etc/slack-cma-bridge/bridge.env`, confirm `AGENTS_CONFIG=/opt/slack-cma-bridge/agents.yaml`, confirm `ls /opt/slack-cma-bridge/agents.yaml` exists |
| Service starts, but Bolt log says `slack_bolt.App` ⚡ then disconnects | `SLACK_APP_TOKEN` is wrong, or missing `connections:write` scope | Regenerate the App-Level Token in Slack app settings, paste into `bridge.env`, restart |
| `@mention` produces no log line | Bot isn't invited to that channel | `/invite @Relay` |
| Bot replies in a new thread but ignores follow-ups without re-tagging | `channels:history` scope or `message.channels` event missing | Slack app → **Features** → **App Manifest** — re-apply the manifest from this repo; reinstall app; update `SLACK_BOT_TOKEN` in `bridge.env` |
| Slash command fails with `invalid_thread_ts` | Outdated `bridge.py` (pre-fix) on the box | `sudo /opt/slack-cma-bridge/deploy/redeploy.sh` to pull the current `main` |
| Bot replies `_(no reply from agent)_` every time | CMA SDK event names changed, or session is blocked on a tool confirmation | Set `LOG_EVENT_TYPES=1` in `bridge.env`, restart, send a test message, inspect what event types arrive in `journalctl`. The known terminal event is `session.status_idle` with `stop_reason.type ≠ "requires_action"` |
| Bot replies `_(agent X errored: ConnectionError…)_` | Outbound network blocked, or Anthropic API outage | `curl -I https://api.anthropic.com` from the box. If that fails, check Lightsail outbound networking and DNS. |
| `redeploy.sh` exits silently every cron run | No commits since last run — this is healthy | `git -C /opt/slack-cma-bridge rev-parse HEAD` to see what the box is on |
| `redeploy.sh` says `Permission denied` | Script not executable | `sudo chmod +x /opt/slack-cma-bridge/deploy/*.sh` |
| `chat_update` falls back to `postMessage` chunks | Agent reply exceeded Slack's 40000-char message limit | Working as designed — long replies post as threaded follow-ups |
| Service goes into a restart loop | Likely a Python import error after a bad merge | `sudo journalctl -u slack-cma-bridge -n 50 --no-pager` to read the traceback; fix in the repo and push |

## 11. Limitations and the path to production

Things you'd build next if this stopped being a single-workspace demo:

| Capability | What it takes |
|---|---|
| **Multi-workspace** | OAuth install flow, install table (Postgres), per-workspace token storage, per-workspace process or shared process with team-level routing. Roughly: a Powerloom-class platform. |
| **User identity** | Map Slack user IDs to internal user records; threading context through to the CMA via session `metadata`; per-user audit log. |
| **RBAC** | Allow/deny specific slugs by Slack user or group; a permissions config in `agents.yaml`. |
| **Approval gates** | Use CMA's `always_ask` permission policy + Slack interactive block buttons to gate sensitive tool calls. |
| **Observability** | OTel traces from the bridge, prometheus metrics on session counts/durations/token usage, CloudWatch log streams instead of journald. |
| **High availability** | Two Lightsail instances behind a static IP failover, or move to ECS Fargate with autoscaling. Note that Slack Socket Mode permits multiple connections per app — they load-balance — so just running two processes works for read-side HA. |
| **Token storage** | Move secrets from a flat env file into AWS Secrets Manager or SSM Parameter Store, with rotation. |
| **Repo private** | If the repo can't be public, use a GitHub deploy key on the box, or fetch via a PAT in the Lightsail user's git credentials. |

---

## 12. Cost reference

| Item | Cost | Notes |
|---|---|---|
| Lightsail nano_3_0 instance | **$3.50/mo** | Includes 1 TB transfer. Counted by the hour at $0.005/hr. |
| Lightsail static IP (if attached) | **Free** | Only billed when detached from a running instance. |
| Lightsail snapshots | **$0.05/GB-month** | Optional. A 20 GB instance snapshot ~ $1/mo per snapshot. |
| Anthropic API usage | **Variable** | At Sonnet 4.6 pricing, ~$0.01–0.05 per typical Slack turn. Long research replies (Vanguard with web_search) can be ~$0.20+ per turn. Cache hits within a session are cheap. |
| Domain / TLS / load balancer | **$0** | None of these are used. |
| Total fixed | **~$3.50/mo** | Pre-usage |

Worst-case typical: $3.50/mo infra + $20–50/mo Anthropic if the workspace is active. Tighten by capping the number of agents people can address, or by routing untagged traffic to a Haiku-based agent.

---

## 13. Quick reference card

```
# SSH in
ssh -i .lightsail/relay ubuntu@<PUBLIC-IP>

# Tail live logs
sudo journalctl -u slack-cma-bridge -f

# Tail redeploy log
sudo tail -f /var/log/slack-cma-bridge-redeploy.log

# Force a redeploy now (skip cron's 5-min wait)
sudo /opt/slack-cma-bridge/deploy/redeploy.sh

# Service control
sudo systemctl {start,stop,restart,status} slack-cma-bridge

# Edit secrets (then restart)
sudo vi /etc/slack-cma-bridge/bridge.env
sudo systemctl restart slack-cma-bridge

# Edit agents (DO NOT do this on the box — edit in repo, push)
# (local) edit agents.yaml, git push
# (on box) sudo /opt/slack-cma-bridge/deploy/redeploy.sh
```

---

*Generated for Relay v1.0 — May 2026. The source for this runbook is in `docs/RUNBOOK.md` in the repo. Updates land there.*